

Project: Build a Data Toolkit in Python

One project, six levels, easy to hard. By the end you'll have used the Python that data / ML / AI engineers actually use — and, more importantly, learned to design the logic yourself.

What this project is

You're building a small toolkit that takes a **messy real-world dataset** and turns it into clean, usable data with a summary report. This is exactly the shape of real data/ML work: data always arrives dirty, and step one of every project is making it usable.

File you're given: **messy_people.csv** — 72 rows of people (id, name, age, city, score). It is **deliberately messy**: missing values, some ages typed as words, duplicate ids, and a few completely blank rows. Cleaning it is part of the job.

How the levels work

Each level teaches one core Python concept and adds one capability to your toolkit. **Early levels give you more guidance. Later levels give you only the problem — you design the solution.** That shift is on purpose: by the end you should be deriving the logic yourself, not following steps. Finish a level fully before moving on.

Level 0 — Load and look (warm-up)

Concept: file handling, lists, dictionaries, loops.

Read the CSV. Store each row as a dictionary (keys: id, name, age, city, score). Put them all in a list. Print how many rows you loaded and print the first 3.

Guidance: use `csv.DictReader`. This level is mostly given — the point is just getting the data into memory in a shape you can work with.

Level 1 — A Record class (OOP begins)

Concept: classes, `__init__`, methods, `@dataclass`.

Instead of loose dictionaries, make a **Record** class — one object per person. First write it as a normal class with `__init__`. Then rewrite it as a `@dataclass` and notice how much shorter it is. Give it a method like `is_valid()` that returns `True` if the row has all its fields filled in.

Guidance: we tell you to make a `Record` class and a validity method. **You decide** what "valid" means and how the method checks it. That decision is yours — that's the first bit of logic you derive.

Level 2 — Clean the data (deriving logic)

Concept: error handling (`try/except`), type conversion, list comprehensions.

The data is dirty. Build cleaning logic that: skips fully blank rows, handles ages that are words or missing, handles missing scores, and removes duplicate ids. Each cleaned row should end up as a valid `Record`.

Guidance: we tell you *what* is wrong with the data. **You decide how to handle each problem** — skip the row? use a default? Which is right for age vs. score vs. a duplicate id? There is no single answer; defend your choices in the meeting. This is the core logic-design level.

Do NOT let a bad row crash your program. A crash on one bad row means one bad row stopped the whole dataset — handle it, don't die on it.

Level 3 — A Dataset class + streaming (generators, context managers)

Concept: generators (`yield`), context managers (`with`), type hints.

Wrap everything in a **Dataset** class that holds many Records and knows how to load, clean, and iterate over them. Add a generator method that yields one clean record at a time instead of returning a giant list — so you could process a huge file without loading it all into memory. Add type hints to your functions.

Guidance: minimal now. You're told the class name and the concepts to use. **You design the methods**, what they take, what they return. Ask yourself why streaming one-at-a-time matters for big data (this exact idea comes back in the real scanner project later).

Level 4 — Analyze + a decorator

Concept: writing a decorator, aggregation logic.

Add analysis methods: average score, count of people per city, oldest/youngest, etc. Then write a **decorator** — for example `@log_time` or `@log_call` — and put it on your analysis methods so each one prints its name (and how long it took) when it runs.

Guidance: only the problem is given. **You work out** what stats are worth computing and how to write the decorator. Writing your first decorator is fiddly — that's expected. Understand what it's doing: a function that wraps another function.

Level 5 — Report + package it (design it yourself)

Concept: modules/packages, pulling it all together, clean structure.

Produce a summary report (printed or saved to a text/JSON file): rows loaded, rows dropped in cleaning and why, and your key stats. Split your code across sensible files/modules instead of one big script.

Guidance: almost none. You're handed the goal — "clean data in, useful report out, well-organised code" — and you design the whole thing. This level is the real test of whether you can derive structure on your own, which is the skill this project exists to build.

What you must be able to explain (for every level)

For each important piece of code you write, be ready to answer in the meeting:

- Why is this here? What job does it do?
- What does it take in, and what does it give back?
- What could go wrong with it, and what did you do about that?
- How did you check it actually works?

Writing code that runs is half the task. Being able to explain *why* it's built that way is the half that turns it into real skill. If you can't explain a line, you don't own it yet.

Rules

- Standard library only for the logic (csv, json, dataclasses, functools are fine). No pandas doing the cleaning for you — the point is you write the logic.
- Commit after each level, on your branch, with a message saying which level. Open a pull request.
- Stuck? Post in Discord: what you tried, the exact error, what you expected. Design questions ("how should I handle missing age?") are great to discuss — that's the learning.

Done when

All six levels work, your code is split into clean modules, and you can run it on `messy_people.csv` and get a clean report out. And you can walk through any part of it in the meeting and explain why you built it that

way.